Unsupervised Deep Learning and Generative Systems

NECKER

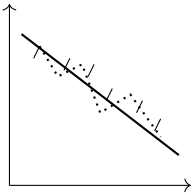
April 8, 2025

Contents

1 dimensionality reduction

In Machine Learning and multivariate statistical analysis, **dimensionality reduction** consists of the process of transforming a high-dimensional dataset $X \in \mathbb{R}^{N \times D}$ into a lower-dimensional representation $Z \in \mathbb{R}^{N \times d}$ (with $d \ll D$), while preserving the most significant portion of the original information structure.

The need for such spatial compression arises from the necessity to mitigate the increase in features, which determines an exponential dispersion of points in the geometric space, making mathematical models unstable, computationally expensive, and prone to *overfitting*.



1.1 Geometric-Algorithmic Analysis of PCA: 2D to 1D Reduction with Noise Collapse

To understand the causal link between algebraic transformations and the geometric collapse of superfluous dimensions, let us analyze the dimensionality reduction of a dataset $X \in \mathbb{R}^{3\times 2}$ composed of 3 non-aligned observations in Cartesian space ($D = 2$), with the goal of compressing them onto a line ($d = 1$).

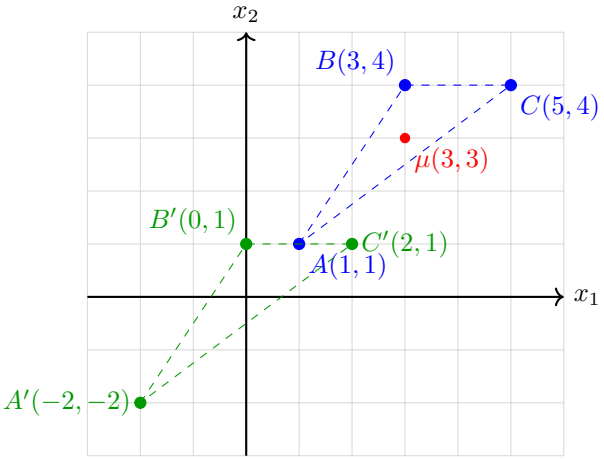
$$X = \begin{bmatrix} 1 & 1 \\ 3 & 4 \\ 5 & 4 \end{bmatrix} \tag{1}$$

1.1.1 Step 1: Data Centering and Translation of the Origin

The geometric barycenter of the cloud is calculated using the vector of sample means $\mu = [\mu_1, \mu_2] = [3, 3]$. Subtracting μ from each coordinate generates the centered data matrix $\bar{X}$:

$$\bar{X} = X - \mathbf{1}\mu = \begin{bmatrix} 1-3 & 1-3 \\ 3-3 & 4-3 \\ 5-3 & 4-3 \end{bmatrix} = \begin{bmatrix} -2 & -2 \\ 0 & 1 \\ 2 & 1 \end{bmatrix} \tag{2}$$

Geometric Meaning: This operation applies a **rigid translation**, moving the center of gravity of the point cloud to the origin of the axes $(0, 0)$. This nullifies the bias vector and allows subsequent transformations to operate as a pure linear operator (rotation).



1.1.2 Step 2: Covariance Matrix (Σ) and Cloud Thickness

The empirical covariance matrix $\Sigma \in \mathbb{R}^{2\times 2}$ maps the spatial dispersion of the centered points:

$$\Sigma = \frac{1}{N-1}\bar{X}^T\bar{X} = \frac{1}{2}\begin{bmatrix} -2 & 0 & 2 \\ -2 & 1 & 1 \end{bmatrix}\begin{bmatrix} -2 & -2 \\ 0 & 1 \\ 2 & 1 \end{bmatrix} = \begin{bmatrix} 4 & 3 \\ 3 & 3 \end{bmatrix} \tag{3}$$

Geometric Meaning: The diagonal elements indicate that the variance along x_1 ($\Sigma_{11} = 4$) is larger than that along x_2 ($\Sigma_{22} = 3$). The positive covariance ($\Sigma_{12} = 3$) indicates an oblique trend. Unlike the degenerate example, the cloud possesses a real two-dimensional "thickness": the points are not aligned but form a triangle (a full dispersion ellipse).

1.1.3 Step 3: Eigenvector Space Extraction (Rotation of the New Axes)

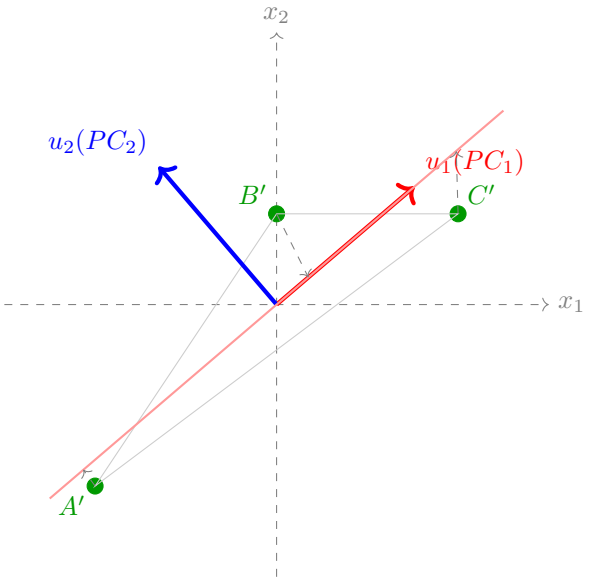
Solving the characteristic equation $\det(\Sigma - \lambda I) = 0$:

$$\det \begin{bmatrix} 4-\lambda & 3 \\ 3 & 3-\lambda \end{bmatrix} = 0 \implies \lambda^2 - 7\lambda + 3 = 0 \tag{4}$$

We obtain the eigenvalues $\lambda_1 \approx 6.54$ (major axis variance) and $\lambda_2 \approx 0.46$ (minor axis variance). Calculating and normalizing the dominant eigenvector for λ_1, we obtain the projection axis u_1:

$$(\Sigma - 6.54I)u_1 = 0 \implies u_1 \approx \begin{bmatrix} 0.76 \\ 0.65 \end{bmatrix} \tag{5}$$

Geometric Meaning: The eigenvectors define the new rotated coordinate system (PC_1, PC_2). The PC_1 axis (directed as u_1) aligns along the maximum extension of the data triangle. The secondary eigenvalue $\lambda_2 = 0.46$ represents the orthogonal thickness, intended as informational noise to be eliminated during dimensional collapse.

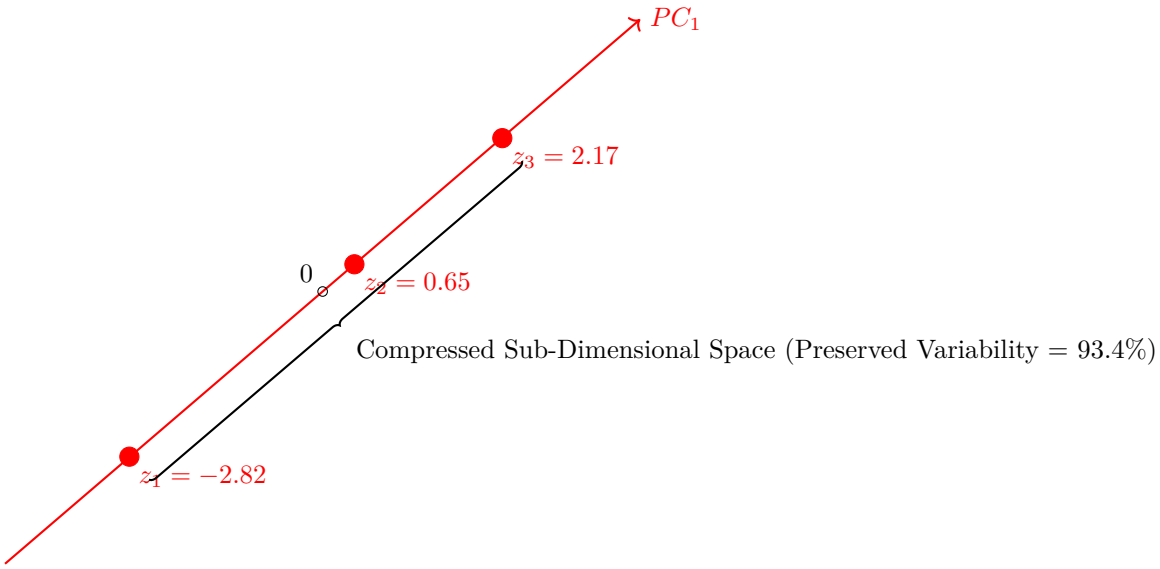


1.1.4 Step 4: Orthogonal Projection and Dimensional Collapse ($d = 1$)

Setting the projection matrix $W = [u_1] \in \mathbb{R}^{2\times 1}$, the linear mapping flattens the two-dimensional data along the single surviving latent axis, generating the one-dimensional dataset $Z \in \mathbb{R}^{3\times 1}$:

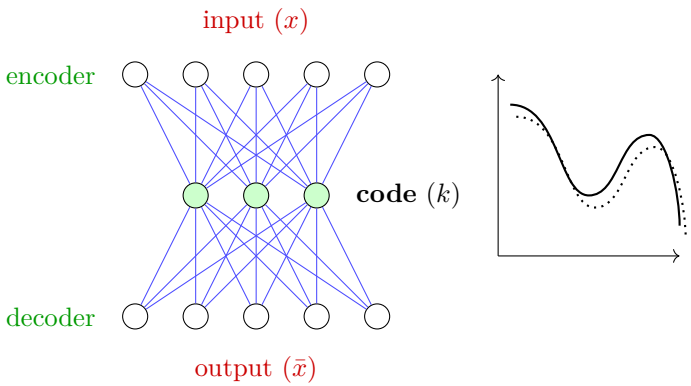
$$Z = \bar{X}W = \begin{bmatrix} -2 & -2 \\ 0 & 1 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} 0.76 \\ 0.65 \end{bmatrix} \approx \begin{bmatrix} -2.82 \\ 0.65 \\ 2.17 \end{bmatrix} \tag{6}$$

Geometric Meaning: This final step performs a **destructive orthogonal projection**. The points A', B', C', originally arranged in a triangle, are projected at a right angle onto the line PC_1. The "thickness" quantified by λ_2 is entirely removed (collapsed to zero). The final dataset no longer has (x_1, x_2) coordinates, but is composed of simple scalars indicating the linear position of the points along the PC_1 track.



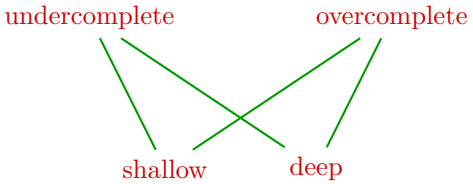
1.2 non-linear autoencoder (ae)

(relu or non-linear functions are used)

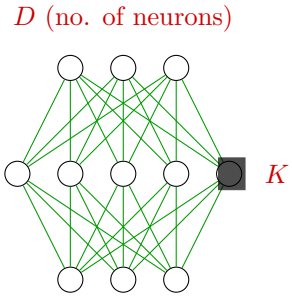


A proper neural network whose purpose is to reduce features and then reconstruct them.
The substantial difference is that it is trained with the input itself. Ex: $L = |x - \bar{x}|^2$

1.3 types of autoencoders



1.3.1 overcomplete: $K > D$



- used for dropout/nullifications in generation
- they learn sparse representations
- or perform denoising

Problem:
In networks where $K < D$, neurons are forced to extract features and specialize; in networks where $K > D$, the network generally finds the mathematical shortcut of setting all weights to 1 and literally doing copy and paste. This phenomenon does not allow the network to specialize neurons for recognizing specific characteristics. To overcome this problem, the solution is to force only some neurons to specialize. This is achieved using **sparse autoencoders**.

sparse autoencoders:
A correction term is added to the loss function to inhibit certain neurons, pushing them to become zero. The logic consists of taking the sum of the outputs of all neurons multiplied by a parameter, and adding it to the actual **Loss** function. By doing so, the network must try to minimize both the initial **Loss** (a low value indicates the image was reconstructed well) and the added parameter (which indicates that the sum of the neuron outputs is very low, hence few active neurons).
WARNING: the initial **Loss** can never be negative, and the added term can never be 0; this means that a small error is introduced and the network will never achieve 100% accuracy.

$$L = \overbrace{\frac{1}{n}\sum_{i=1}^{n} |x^i - \bar{x}^i|^2}^{L} + \lambda \sum_j |h_j|$$

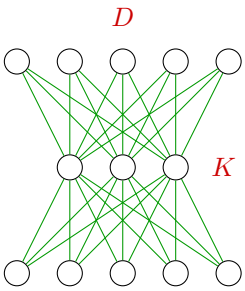
$\lambda \sum_j |h_j|$**:**

- h_j: output of the neuron
- $\sum$: summation of all outputs
- λ: hyperparameter (like the learning rate) that determines how much to penalize

denoising autoencoders:
Once the specialization problem is solved, we encounter the denoising problem. Indeed, if we train the network to reconstruct images starting from a perfect input, when a blurry or imperfect image is provided, the model will struggle to reconstruct it correctly. To make the network immune to this problem, the input is corrupted to train the network to find the truly useful data information needed to reconstruct the input.

$$X \to \text{noise } \blacksquare \to X' \to \text{encoder} \to K \to \text{decoder} \to \bar{X} \quad \Big| \quad L = |X - \bar{X}|^2$$

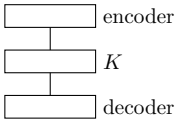
1.3.2 undercomplete: $K < D$



They serve to reduce dimensionality by extracting the most important features.

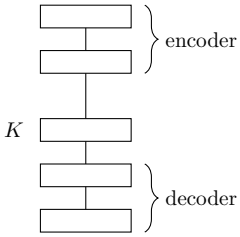
1.4 shallow and deep

shallow:



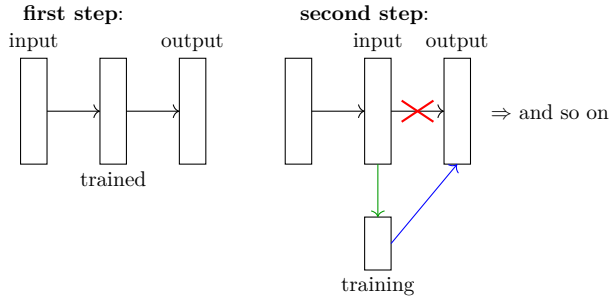
They learn only a few non-linear transformations.

deep:



Deeper (more layers), they learn much more complex characteristics (more accurate).

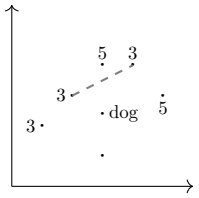
training:
Nowadays, they are trained like a normal network by carefully handling weight initialization to avoid exploding or vanishing gradients. In the past, however, the network was trained in blocks:



In practice, one block was trained, its weights were then frozen, and that block became the input for the newly added one.

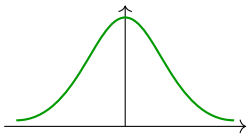
2 variational autoencoder (vae)

Variational autoencoders (**vae**) allow us to better generate the latent space (code $K \rightarrow$ features inside K). In fact, with standard (**ae**) we learn a single Z for each image, often resulting in very sparse and disorganized representations.



The idea is that without a structured method, the system learns in a disordered manner.

Instead, through (vae), I try to guide the network to learn a representation of features clustered around zero (mean around zero and with variance 1 which stabilizes the point).



prior ($P(z)$)

The **prior** indicates the model it must follow; it should converge to a similar curve.

This also enables us to generate completely different images by moving slightly within the space (since all images are now ordered).

$$P(z) \sim \mathcal{N}(\underbrace{0}_{\text{zero mean}}, \overbrace{I}^{\text{identity covariance matrix}})$$

2.1 The Limitation of Deterministic Autoencoders (AE)

The problem arises during the generation phase: when sampling a point in the vast empty regions between data points, the decoder receives a random input pattern and generates pure visual noise. It lacks the fundamental property of **continuity**.

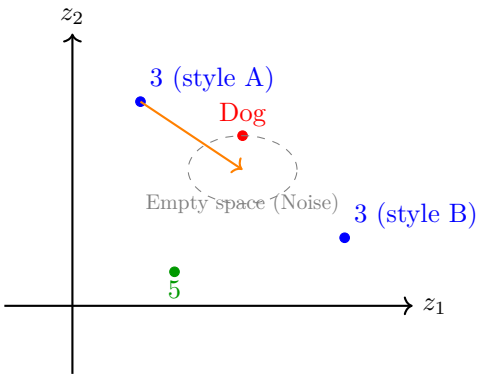


Figure 1: Latent space of a classical AE. The data consists of isolated points separated by vast empty spaces lacking any generative meaning.

2.2 The VAE Solution: Probabilistic Thickness and Sampling

The VAE resolves the empty space issue by transforming discrete points into **probability clouds** (Gaussian distributions). The encoder no longer outputs a fixed vector z, but rather two vectors of parameters: the mean $\mu(x)$ and the variance $\sigma^2(x)$.

The conditional distribution of the latent variables becomes:

$$q(z|x) \sim \mathcal{N}(\mu(x), \sigma^2(x)I) \tag{7}$$

To allow training via backpropagation, the **Reparameterization Trick** is introduced. The actual vector z fed into the decoder is sampled by adding a stochastic noise vector $\varepsilon \sim \mathcal{N}(0, I)$:

$$z = \mu(x) + \sigma(x) \odot \varepsilon \tag{8}$$

Geometric meaning: The network is forced to reconstruct the input starting from a random point *inside* the cloud. This forces the clouds to expand and fill the empty space, ensuring that every point in the neighborhood of the mean retains a valid semantic meaning for the decoder.

2.3 The Functional Loss: A Geometric Tug-of-War

The engineering goal of the VAE is not to create separate disordered distributions, but to force the entire latent space to organize itself into a **topological continuum** governed by a standard Gaussian (Prior) $P(z) \sim \mathcal{N}(0, I)$.

The VAE Loss function is composed of two competing forces:

$$\mathcal{L}_{\text{VAE}} = \underbrace{-\mathbb{E}_{z\sim q(z|x)}\big[\log p(x|z)\big]}_{\text{Reconstruction Term}} + \underbrace{\mathcal{D}_{\text{KL}}\Big(q(z|x) \parallel \mathcal{N}(0, I)\Big)}_{\text{Kullback-Leibler Divergence}}$$

Anatomy of the Functional Loss

To understand the optimization dynamics of the VAE, we must isolate the individual mathematical operators that compose the cost functional:

- $\mathcal{L}_{\text{VAE}}$ **(Overall Loss):** Represents the objective function (*Evidence Lower Bound*, or ELBO, rewritten for minimization purposes). The algorithm applies stochastic gradient descent to contract this global numerical value toward its asymptotic lower bound.
- $-\mathbb{E}_{z\sim q(z|x)}\big[\log p(x|z)\big]$ **(Expected Reconstruction Log-Likelihood):**
  - **The Minus Sign** $(-)$**:** The log will be negative, and we know that if the **Loss** is high, then the model is making large mistakes; therefore, the minus sign is added to make the total value positive.
  - **The Operator** $\mathbb{E}_{z\sim q(z|x)}$**:** Indicates the expected value (statistical average) computed over all possible latent vectors z sampled from the internal distribution $q(z|x)$ generated by the encoder.
  - **The Term** $\log p(x|z)$**:** Represents the log-likelihood of the decoder reconstructing the exact input x given the code z. From a practical standpoint, it translates to Mean Squared Error (MSE) for continuous data or *Cross-Entropy* for binary data.
  - this part of the formula is equivalent to the **MSE**; if you want to see the proof of equality, explore further.
- $\mathcal{D}_{\text{KL}}\Big(q(z|x) \parallel \mathcal{N}(0, I)\Big)$ **(Kullback-Leibler Penalty):**
  - **The Operator** $\mathcal{D}_{\text{KL}}(\cdot \parallel \cdot)$**:** Is an asymmetric measure of statistical divergence (relative entropy) that quantifies the informational discrepancy between two probability distributions.
  - **The Members** $q(z|x)$ **and** $\mathcal{N}(0, I)$**:** It measures how much the local Gaussian created by the encoder for a specific image deviates from a centralized standard Gaussian at the origin with unit variance.
  - **Regularizing Function:** This block acts as a geometric compaction constraint. It prevents the means μ from diverging to infinity and forces the variances σ^2 to stabilize at unity, ensuring the absence of gaps in the latent space.

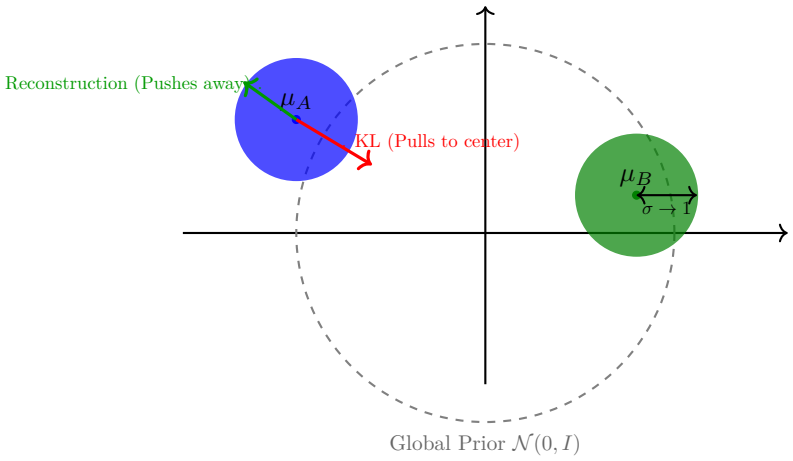


Figure 2: The dynamic balance: The KL Divergence acts as a centripetal force, pulling the means toward the origin and expanding the variance to 1. The Reconstruction Error acts as a centrifugal force, separating visually incompatible clusters to prevent ambiguity.

2.4 Unsupervised Cluster Formation and Interpolation

A common misconception is that the KL explicitly identifies classes or that the architecture requires labels to group similar images together. In reality, clustering is an **emergent and spontaneous property**.

1. **Pixel Invariance:** The Decoder requires that latent vectors with high pixel overlap (e.g., two styles of the number "3") be mapped close to each other, so it can use the same weights for reconstruction, thereby reducing the loss.
2. **Isotropic Regularization:** The KL forces all these Gaussians to have a near-zero mean and wide variance ($\sigma \approx 1$).

Being forced to occupy the same central region and having expanded radii, the distributions of akin classes are **geometrically constrained to overlap**.

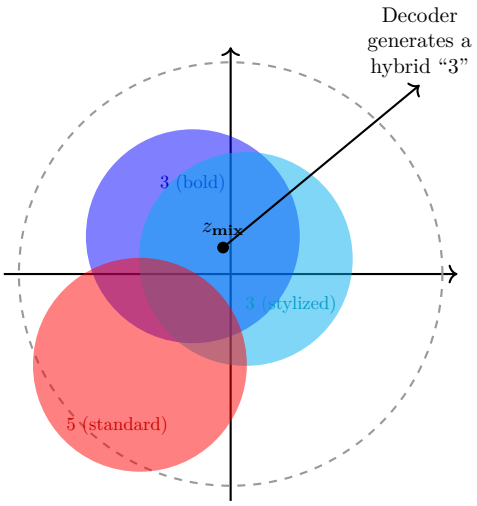


Figure 3: Continuous VAE Latent Space (The Mosaic Effect). The Gaussians are not separated but interlocked. By sampling in the intersection area (z_{mix}), the network performs a semantic interpolation, blending the morphological features of the overlapping distributions.

Analytical Conclusion: The latent space of a VAE ceases to be a discrete archive and becomes a unified genetic matrix. By sampling inside areas of probabilistic overlap, the algorithm does not fail, but instead performs a **morphological synthesis** between the pixel gradients of the two interacting distributions, generating fluid features and entirely unique images.

WARNING: if we extract a point between two Gaussians, we will have uncertainty regarding which number to generate; since they are Gaussians, unless the point is exactly in the middle, the probability will bias heavily toward one side over the other.

2.5 The Mechanics of Weight Updates and Gradient Flow

To understand how the variational loss $\mathcal{L}_{\text{VAE}}$ modifies synaptic parameters, we must map the gradient flow within the computational graph. Optimization takes place simultaneously on two fronts.

2.5.1 The Computational Graph and the Reparameterization Trick

In a variational architecture, standard backpropagation would encounter a non-differentiable stochastic barrier at the sampling step of the latent vector z. Isolating the noise via the *Reparameterization Trick* ($z = \mu + \sigma \odot \varepsilon$) restructures the graph, making it fully differentiable.

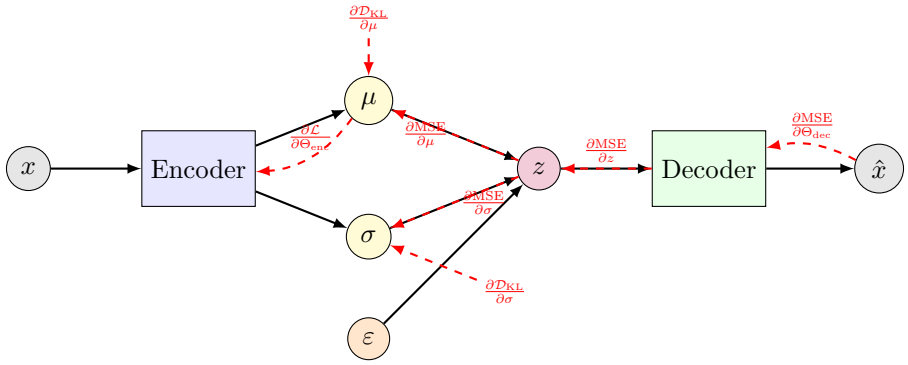


Figure 4: Deterministic gradient flow (red dashed lines). Thanks to the reparameterization trick, the reconstruction error flows freely from the decoder back to the encoder through the continuous channel of z.

2.5.2 Mathematical Derivation of Parameter Gradients

Gradient descent updates the set of weights and biases of the encoder (Θ_{enc}) and the decoder (Θ_{dec}) based on the additive decomposition of the total loss $\mathcal{L}_{\text{total}} = \text{MSE} + \mathcal{D}_{\text{KL}}$.

1. **Decoder Update (Θ_{dec}):** The statistical regularization block $\mathcal{D}_{\text{KL}}$ is computed upstream of the sampling step and does not depend on the decoder parameters. Therefore, the partial derivative with respect to the weights W_{dec} treats the KL term as a zero constant:

$$\frac{\partial \mathcal{L}_{\text{total}}}{\partial W_{\text{dec}}} = \frac{\partial \text{MSE}}{\partial W_{\text{dec}}} + \frac{\partial \mathcal{D}_{\text{KL}}}{\partial W_{\text{dec}}} = \frac{\partial \text{MSE}}{\partial W_{\text{dec}}} + 0 \tag{9}$$

   The correction of the decoder weights follows the classic stochastic optimization algorithm governed by the learning rate η:

$$W_{\text{dec}}^{(t+1)} = W_{\text{dec}}^{(t)} - \eta \cdot \frac{\partial \text{MSE}}{\partial W_{\text{dec}}^{(t)}} \tag{10}$$

2. **Encoder Update (Θ_{enc}):** The encoder controls the shape of the latent space, directly influencing both the geometric layout of the means and variances (under the jurisdiction of the KL) and the sampled point z that determines the reconstruction quality (MSE). The total gradient with respect to W_{enc} is a linear combination of these two forces:

$$\frac{\partial \mathcal{L}_{\text{total}}}{\partial W_{\text{enc}}} = \frac{\partial \text{MSE}}{\partial W_{\text{enc}}} + \frac{\partial \mathcal{D}_{\text{KL}}}{\partial W_{\text{enc}}} \tag{11}$$

   Applying the chain rule to the first term and expanding the closed form of the KL derivative with respect to the local distributional parameters (μ_j, σ_j^2), we obtain the exact components of the gradient:

$$\frac{\partial \mathcal{L}_{\text{total}}}{\partial W_{\text{enc}}} = \sum_{j=1}^{K} \left(\frac{\partial \text{MSE}}{\partial z_j} \cdot \frac{\partial z_j}{\partial W_{\text{enc}}} \right) + \frac{\partial}{\partial W_{\text{enc}}} \left[-\frac{1}{2} \sum_{j=1}^{K} \left(1 + \log(\sigma_j^2) - \mu_j^2 - \sigma_j^2\right) \right] \tag{12}$$

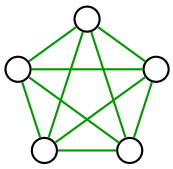
2.5.3 The Dynamic Balance of Forces on Encoder Neurons

- **The KL gradient** generates a thrust proportional to $-\mu_j$ and $(1-\sigma_j^2)$. If a mean drifts away from zero or a variance collapses, this derivative forces the encoder weights to adjust their configuration to pull the distribution back toward the spherical shape of the prior.
- **The MSE gradient** propagates backward through the decoder matrices, quantifying the pixel-level error. It hits the encoder parameters, demanding that the means μ of dissimilar classes move in opposite directions, countering the compaction of the KL to preserve semantic separability.

The final encoder update occurs by accumulating the contributions of both force vectors:

$$W_{\text{enc}}^{(t+1)} = W_{\text{enc}}^{(t)} - \eta \cdot \left(\frac{\partial \text{MSE}}{\partial W_{\text{enc}}^{(t)}} + \frac{\partial \mathcal{D}_{\text{KL}}}{\partial W_{\text{enc}}^{(t)}} \right) \tag{13}$$

3 hopfield networks



- networks composed of a single interconnected layer
- symmetric connections: $W_{ji} = W_{ij}$
- binary neurons: $\{0, 1\}$ active/inactive or $\{-1, 1\}$ functions used for symmetry

Hopfield networks are capable of storing various neuron configurations and are highly useful for reconstructing data starting from fragmented information. To understand this better, imagine a subset of correctly activated neurons; they exert an attractive pull on corrupted or silent units (forcing them to flip states to conform to the memorized pattern).

3.1 Weight Updates:

During each cycle, all neurons are updated but one at a time in a random order:

- neuron input: $\boxed{z = \sum_{j \neq i} W_{ji} Y_j(t) - b_i}$

The summation of all neuron states weighted by their connection strength at time t. b_i = subtracted bias, introduced to make inactive states less prone to easily firing.

3.2 Local State Transition Dynamics

The activation rule defines the behavior of the i-th neuron based on the net post-synaptic potential received at time t:

$$Y_i(t+1) = \begin{cases} 1 & \text{if } \displaystyle\sum_{j \neq i} W_{ji} Y_j(t) - b_i > 0 \\ 0 & \text{if } \displaystyle\sum_{j \neq i} W_{ji} Y_j(t) - b_i \leq 0 \end{cases} \tag{14}$$

To understand the microscopic evolution of the system, we define the net stimulus as $I_i(t) = \sum_{j \neq i} W_{ji} Y_j(t) - b_i$. We can map the four fundamental logical transitions that govern associative memory convergence:

- **Preservation of the Active State (Stability):** If $Y_i(t) = 1$ and the net stimulus $I_i(t) > 0 \longrightarrow Y_i(t+1) = 1$. The neuron confirms its activation as it is consistent with the context of the weights.
- **Preservation of the Silent State (Inactivity):** If $Y_i(t) = 0$ and the net stimulus $I_i(t) \leq 0 \longrightarrow Y_i(t+1) = 0$. The neuron stays off, preserving the pattern's informational gap.
- **Dynamic Inhibition (Noise Correction):** If $Y_i(t) = 1$ but the net stimulus $I_i(t) \leq 0 \longrightarrow Y_i(t+1) = 0$. The neuron was in a state of false activation due to noise; collective pressure dictates its shutdown.
- **Dynamic Excitation (Pattern Completion):** If $Y_i(t) = 0$ but the net stimulus $I_i(t) > 0 \longrightarrow Y_i(t+1) = 1$. The neuron experiences a forced turn-on induced by adjacent units to reconstruct the missing fraction of the memory.

3.3 energy function: (applies to symmetric networks $\{-1, 1\}$)

It helps us understand how much energy the network has (how far it is from a stable network configuration). Furthermore, since it is monotonically decreasing, it guarantees convergence.

$$\boxed{E(Y) = -\frac{1}{2}\sum_{i=1}^{N}\sum_{j=1}^{N} W_{ij}Y_iY_j + \sum_{i=1}^{N} b_iY_i}$$

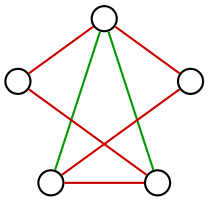
- $E(Y)$ **(Global Energy of the System):** Is a scalar function bounded from below. It functions like a "topographic map": the patterns the network needs to memorize correspond to the lowest points of this surface (valleys or basins of attraction). The lower the value, the closer the current neuron configuration is to a stable memory.
- **The Scale Factor $-\frac{1}{2}$:**
  - **The Minus Sign $(-)$:** Determines the sign of the interaction. If two neurons are linked by a positive weight ($W_{ij} > 0$) and both are active ($Y_i = 1, Y_j = 1$), their product is positive. The preceding minus sign transforms this stability into a negative energy value, lowering the global cost and rewarding pattern consistency.
  - **The Coefficient $\frac{1}{2}$:** Compensates for the double-counting in the double summation. Since the network has symmetric connections ($W_{ij} = W_{ji}$), the link between neuron i and neuron j is processed twice (once as the pair (i, j) and once as (j, i)). Multiplying by half normalizes the value, computing the actual energy of each single synapse without duplications.
- $\sum_{i=1}^{N}\sum_{j=1}^{N} W_{ij}Y_iY_j$ **(Mutual or Quadratic Interaction Term):**
  - **The Double Summation:** Performs a complete combinatorial scan of all possible pairs of neurons present in the network (from 1 to N).
  - **The Product $W_{ij}Y_iY_j$:** Represents the internal energy of the connection. If the weight matches the state of the neurons (e.g., positive weight and both neurons active, or negative weight and one neuron off), the system experiences a local energy reduction. If there is a mismatch (e.g., both neurons active but joined by a negative inhibitory link), the energy increases, indicating structural instability.
- $\sum_{i=1}^{N} b_iY_i$ **(Linear Potential Term or Bias Contribution):**
  - **The Vector b_i:** Represents the internal activation threshold (*bias*) isolated for each individual neuron. It acts as an external magnetic field independent of the node interactions.
  - **The Linear Summation:** Computes the static energetic cost of activation. Because this term is added (unlike the quadratic block preceded by the minus sign), the presence of a positive bias $b_i > 0$ paired with an active neuron ($Y_i = 1$) linearly increases the overall energy. Consequently, the bias operates as a restraining or resistive force, making the activation of that neuron inherently more costly and favoring system inactivity unless strong overriding synaptic thrusts are present.

3.4 learning: (hebbian rule)

Classic backpropagation is not used here; instead, we rely on Hebb's rule.

$$\boxed{W_{ji} = Y_j Y_i} \quad \text{base rule (valid for a single pattern)}$$

This implies that if 2 neurons share the same activation, their connection weight will be positive (excitatory); otherwise, it will be negative (inhibitory).



general rule (multiple patterns): *(intended as the various local minima of a function)*

$$\boxed{W_{ij} = \frac{1}{N} \sum_{p \in \{1,P\}} Y_i^p Y_j^p}$$

- **The Normalization Coefficient $\frac{1}{N}$:**
  - **The Term N:** Represents the total number of neurons making up the entire network architecture.
  - **Scaling Function:** Divides the sum by the number of nodes to prevent synaptic weight values from expanding uncontrollably as the size of the network grows. It maintains weights within a stable numeric range, ensuring that energy gradients remain proportionate and do not saturate the system.
- $\sum_{p \in \{1,P\}}$ **(Memory Accumulation Operator):**
  - **The Set $\{1, P\}$:** Represents the collection of all P fundamental patterns (images, vectors, ideal configurations) that the network must memorize simultaneously.
  - **The Summation:** Implements the principle of memory superposition. The network does not learn one pattern at a time by overwriting the previous one; conversely, the contributions of all P vectors are **linearly summed** inside the exact same weight matrix W. Each synapse accumulates the history of co-activations across all patterns in the dataset.
- **The Product $Y_i^p Y_j^p$ (Hebbian Correlation for Pattern p):**
  - **The Terms Y_i^p and Y_j^p:** Represent the target state (on or off) of neuron i and neuron j specifically within the p-th pattern.
  - **Product Logic (If states are binarized as $\{-1, 1\}$):** If in pattern p the two neurons share the same state (both 1 or both -1), their product is $+1$, so the weight W_{ij} increases (cooperation). If they have opposite states, their product is -1, so the weight decreases (inhibition).
  - **Product Logic (If states are binarized as $\{0, 1\}$):** The product outputs 1 only if both neurons are active ($1 \cdot 1 = 1$). If one or both are off (0), the product drops to zero (0). This means that the synaptic link is actively reinforced only to drive the co-firing effect of the units, faithfully mirroring Hebb's biological postulate: *"Neurons that fire together, wire together"*.

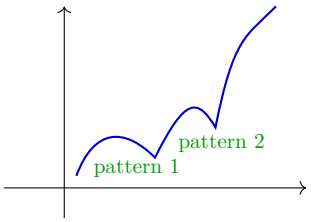
for example with 3 patterns:

$$W_{ij} = \frac{1}{N} \sum_{p=1}^{3} Y_i^p Y_j^p = \frac{1}{N} (Y_i^1 Y_j^1 + Y_i^2 Y_j^2 + Y_i^3 Y_j^3)$$

All three patterns (pattern 1, pattern 2, and pattern 3) are "compressed" and fused together inside a single

large weight matrix W.

When the matrix W has accumulated the 3 patterns through that summation, the global energy surface of the network is reshaped. Imagine the energy surface as a stretched rubber sheet: the Hebbian formula has carved out three large hollows (valleys) corresponding to the clean target patterns. These valleys are mathematically called Stable Attractors.



By introducing a noisy pattern, within a few iterations the energy stabilizes, pulling us down into a basin (a memorized pattern).
If there are too many patterns, memorizing them all becomes difficult. solution: raise the curve's energy to make individual patterns stand out sharply.